Manipulador

Práctica N°1

Nicol Juliana Tuta Niño
Escuela Colombiana de Ingeniería
Julio Garavito
Bogotá, Colombia
nicol.tuta-n@mail.escuelaing.edu.co

Ángel Fabián Pedreros Almeida
Escuela Colombiana de Ingeniería
Julio Garavito
Bogotá, Colombia
angel.pedreros-a@mail.escuelaing.edu.co

I. Introducción

En la presente actividad se desarrollaron tres programas de control para un robot industrial utilizando el entorno de programación Polyscope, ejecutado mediante contenedores Docker.

El objetivo principal fue implementar trayectorias, control de entradas y salidas digitales, interacción con el usuario mediante mensajes emergentes y simulación de tareas industriales como el pick and place.

Adicionalmente, se exploró la integración con control externo mediante ROS2, permitiendo ampliar las capacidades del sistema robótico hacia entornos colaborativos y de planificación avanzada de trayectorias.

II. Metodología

A. Programa 1:Escritura de letras

Este programa tiene como objetivo simular la escritura de iniciales mediante trayectorias del robot. Antes de ejecutar cada letra, se implementó un mensaje emergente (popup) indicando al usuario las letras a generar. Las primeras fueron **AP** del integrante Ángel Pedreros, y luego **NT** de la integrante Nicol Tuta.

Para su implementación se utilizaron movimientos lineales (moveL) junto con una serie de puntos definidos manualmente (waypoints), los cuales permiten representar la geometría de cada letra. Para cada inicial, se realizó un proceso de enseñanza en el que se definieron múltiples waypoints que corresponden a posiciones específicas del efector final en el espacio, organizados secuencialmente para simular un trazo continuo.

El uso de movimientos moveL permitió generar trayectorias rectas entre los distintos waypoints, lo cual resulta fundamental para simular de forma precisa la escritura, ya que mantiene una trayectoria lineal controlada tanto en posición como en orientación del efector final. Cada segmento de las letras fue construido mediante secuencias de movimientos lineales, y en los cambios de dirección se añadieron puntos intermedios que mejoran la definición de la forma.

Antes de iniciar la trayectoria de cada letra, se implementó un mensaje emergente (popup) dentro de Polyscope que informa al usuario cuál inicial será generada, permitiendo una interacción más clara con el sistema. El flujo del programa consiste en mostrar el mensaje, esperar la confirmación del usuario y posteriormente ejecutar la trayectoria correspondiente.

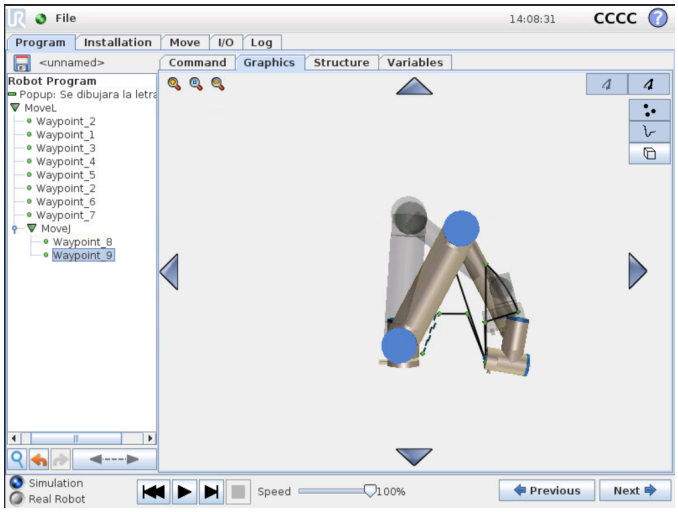

Fig. 1. Imagen del Polyscope escribiendo AP

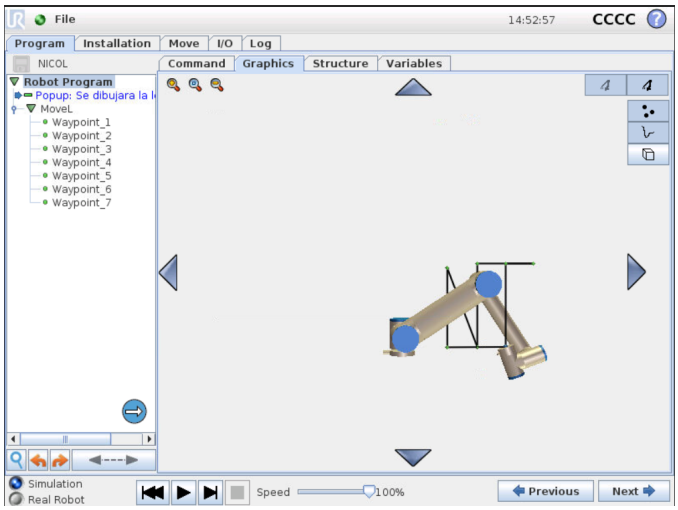

Fig. 2. Imagen del Polyscope escribiendo NT

B. Programa 2: Conexión a External Control

En este programa se implementó la configuración necesaria para permitir el control del robot mediante un sistema externo,

específicamente utilizando ROS2, a través del nodo de *External Control* disponible en Polyscope.

1) Activación de salida digital: Al inicio del programa se configuró una salida digital en estado alto (`DO[0] = ON`). Esta acción permite indicar el comienzo de la ejecución del programa o habilitar algún dispositivo externo, como un indicador o sistema auxiliar.

2) Interacción con el usuario: Se implementó un mensaje emergente (*popup*) para informar al usuario que el control del robot será transferido a un controlador externo. El contenido del *popup* indica que el robot será controlado externamente y que el usuario debe confirmar la acción para continuar.

3) Temporización del sistema: Una vez confirmado el *popup*, se introduce una espera de 5 segundos utilizando una instrucción `Wait`. Esta pausa permite:

4) Configuración del control externo: Posteriormente, se agregó el nodo de *External Control*, el cual permite que el robot sea manipulado desde ROS2.

Para establecer la comunicación, se configuraron las direcciones IP de la siguiente manera:

H

- Controlador externo (ROS2): `172.17.0.1`
- Robot simulado: `172.17.0.2`

Esta configuración permite la comunicación entre el entorno de simulación del robot y el sistema ROS2 mediante una red local. Se puede observar que la conexión fue exitosa debido a que en la intrucción aparece el nombre *robvot*

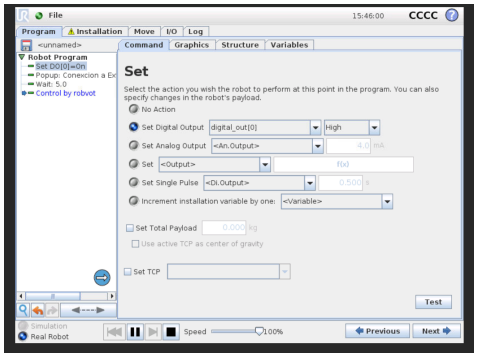

Fig. 3. Conexión exitosa con External Control

Se observa por último la ventana de RViz con el modelo del UR5 funcionando:

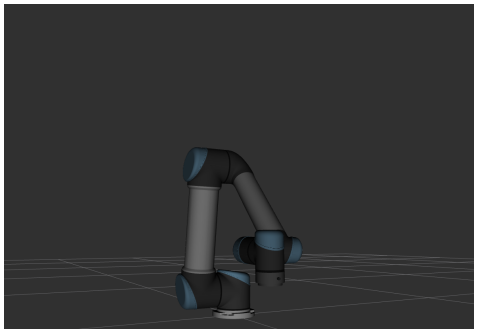

Fig. 4. Ventana de RViz

C. Programa 3: Rutina de Pick and Place

1) Descripción general: La lógica implementada consiste en que el robot se desplace hacia múltiples puntos de recogida y, para cada uno de ellos, ejecute una secuencia estructurada de movimientos que garantizan una manipulación segura del objeto. Posteriormente, el robot transporta el objeto hacia un punto de destino, pasando por posiciones intermedias que aseguran trayectorias libres de colisión.

2) Estructura de la rutina: La rutina se diseñó siguiendo la siguiente secuencia:

- Desplazamiento hacia un punto intermedio superior al punto de recogida.
- Descenso hasta el punto de recogida.
- Activación del efector final mediante una salida digital.
- Retorno al punto intermedio superior.
- Desplazamiento hacia un punto intermedio superior al punto de entrega.
- Descenso al punto de entrega.
- Desactivación del efector final para liberar la carga (*payload*).

Este procedimiento se repite para al menos dos puntos de recogida, los cuales se pueden visualizar en la siguiente imagen:

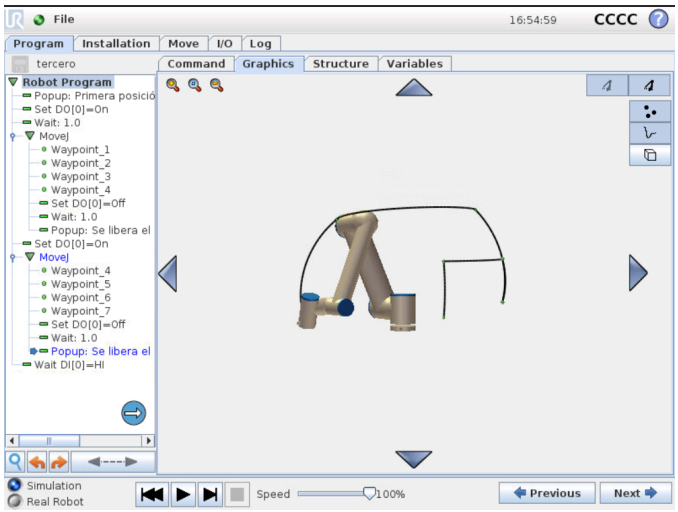

Fig. 5. Polyscope Simulación

3) Uso del comando `MoveJ`: Para la implementación de las trayectorias se utilizó principalmente el comando `MoveJ`. Este tipo de movimiento corresponde a una interpolación en el espacio de las articulaciones del robot.

El uso de `MoveJ` reduce la complejidad computacional al trabajar directamente en el espacio articular y permite movimientos más rápidos en comparación con trayectorias

lineales (`MoveL`). Sin embargo, es importante considerar que `MoveJ` no garantiza trayectorias rectas en el espacio cartesiano, por lo que se hace necesario el uso de puntos intermedios para evitar posibles colisiones con el entorno o con los objetos manipulados.

4) Puntos intermedios y seguridad: El efector final se controló mediante señales digitales que simula la idea de poder abrirlo y cerrarlo:

- Activación (`DO = ON`) en el punto de recogida para sujetar el objeto.
- Desactivación (`DO = OFF`) en el punto de entrega para liberar la carga.

III. Conclusión

A lo largo de la práctica se lograron implementar tres programas en el entorno de Polyscope, ejecutado mediante contenedores Docker. Se evidenció la importancia de la planificación de trayectorias mediante el uso de comandos como moveL y MoveJ, así como la definición adecuada de waypoints, los cuales permiten construir movimientos precisos y adaptados a distintas tareas.

- En el primer programa, se comprendió cómo unión de los puntos facilita la generación de trayectorias controladas para representar formas específicas, como las iniciales, destacando la relevancia de la secuenciación de puntos y la interacción con el usuario mediante mensajes emergentes.
- El segundo programa permitió explorar la integración con sistemas externos a través de ROS 2, evidenciando cómo el nodo de External Control amplía las capacidades del robot al permitir su manipulación desde entornos más avanzados de planificación y control.
- En la rutina de pick and place, se logró implementar una lógica de manipulación de objetos utilizando señales digitales para el control del efector final, junto con trayectorias que incluyen puntos intermedios para garantizar una operación ordenada.

En conjunto, la práctica permitió fortalecer la comprensión del control de robots industriales, la interacción con señales de entrada y salida, y la integración con herramientas externas, consolidando conocimientos fundamentales para aplicaciones reales en la robótica.